

Chapter 6. 임베딩(Word2Vec~)

Chapter 6. 임베딩

6.1 ~ 6.3

Chapter 5. 토큰화 & Chapter 6. 임베딩(~TF-IDF).

6.4 Word2Vec

2013년 구글에서 공개한 임베딩 모델로 단어 간의 유사성을 측정하기 위해 분포 가설을 기반 개발

- 분포 가설: 같은 문맥에서 함께 자주 나타나는 단어들은 서로 유사한 의미 가능성 ↑
 - 분포 가설은 단어 간의 **동시 발생** 확률 분포를 이용해 단어 간의 유사성 측정
- 이러한 가정을 통해 단어의 **분산 표현**을 학습
 - 분산 표현: 단어를 고차원 벡터 공간에 매핑하여 단어의 의미를 담는 것을 의미
- 분포 가설에 따라 단어의 의미는 문맥상 분포적 특성을 통해 나타남
(=유사한 문맥에서 등장하는 단어는 비슷한 벡터 공간상 위치를 가짐)
- 장점
 - 빈도 기반 벡터화 기법에서 발생했던 단어의 의미 정보 저장 한계 극복
 - 대량의 텍스트 데이터에서 단어 간의 관계 파악
 - 벡터 공간상에서 유사한 단어를 군집화, 단어의 의미 정보 효과적으로 표현
 - 다양한 자연어 처리 작업에서 높은 성능
 - 다운스트림 작업에 더 뛰어난 성능

6.4.1 단어 벡터화

- 단어 벡터화 방법은 크게 **희소 표현**, **밀집 표현**으로 나눌 수 있다
- 희소 표현(sparse representation): 원-핫 인코딩, TF-IDF 등의 빈도 기반 방법

표 6.3 단어의 희소 표현

소	0	1	0	0	0
읽고	1	0	0	0	0
외양간	0	0	0	1	0
고친다	0	0	0	0	1

- 대부분의 벡터 요소가 0으로 표현
- 특징
 - 단어 사전의 크기가 커지면 벡터의 크기도 커지므로 공간적 낭비 발생
 - 단어 간의 유사성을 반영하지 못하고, 벡터 간 유사성을 계산하는 데 많은 비용 발생
- 밀집 표현(dense representation): Word2Vec

표 6.4 단어의 밀집 표현

소	0.3914	-0.1749	...	0.5912	0.1321
읽고	-0.2893	0.3814	...	-0.1492	-0.2814
외양간	0.4812	0.1214	...	-0.2745	0.0132
고친다	-0.1314	-0.2809	...	0.2014	0.3016

- 단어를 고정된 크기의 실수 벡터로 표현
- 특징
 - 단어 사전의 크기가 커지더라도 벡터의 크기가 커지지X
 - 벡터 공간상에서 단어 간의 거리를 효과적으로 계산 가능
 - 벡터의 대부분이 0이 아닌 실수 → 효율적인 공간 활용
- 밀집 표현 벡터화는 학습을 통해 단어를 벡터화 → 단어 의미 비교 가능
- 밀집 표현된 벡터를 **단어 임베딩 벡터(Word Embedding Vector)** 라고 한다
- Word2Vec은 밀집 표현을 위해 CBoW와 Skip-gram 두 가지 방법 사용

6.4.2 CBoW(Continuous Bag of Words)

주변에 있는 단어를 가지고 중간에 있는 단어를 예측하는 방법

- 관련 용어 정리

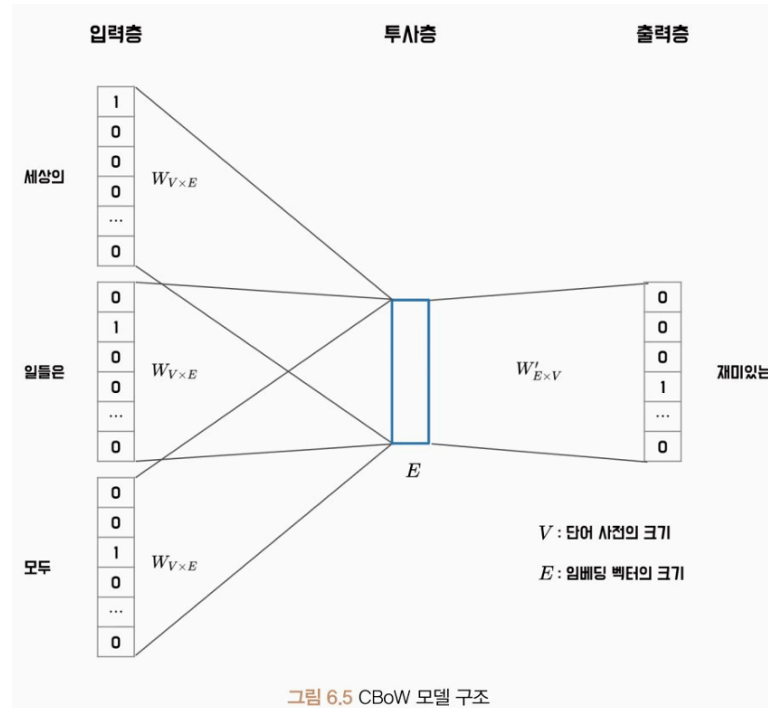
용어	의미
중심 단어(Center Word)	예측해야 할 단어
주변 단어(Context Word)	예측에 사용되는 단어들
윈도(Window)	중심 단어를 맞추기 위해 몇 개의 주변 단어를 고려할지 정하는 범위
슬라이딩 윈도(Sliding Window)	학습을 위해 윈도를 이동해 가며 학습

- CBoW는 슬라이딩 윈도를 사용해 한 번의 학습으로 여러 개의 중심 단어와 그에 대한 주변 단어 학습
- 하나의 입력 문장에 윈도 크기가 2일 때

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(재미있는, 일들은 세상의)
(세상의 재미있는 일들은 모두)밤에 일어난다	(세상의, 일들은, 모두 재미있는)
(세상의 재미있는 일들은 모두 밤에)일어난다	(세상의, 재미있는, 모두, 밤에 일들은)
세상의(재미있는 일들은 모두 밤에 일어난다)	(재미있는, 일들은, 밤에, 일어난다 모두)
세상의 재미있는(일들은 모두 밤에 일어난다)	(일들은, 모두, 일어난다 밤에)
세상의 재미있는 일들은(모두 밤에 일어난다)	(모두, 밤에 일어난다)

그림 6.4 CBoW의 학습 데이터 구성

- 학습 데이터는 (주변 단어 | 중심 단어) 로 구성
- 대량의 말뭉치에서 효율적인 단어 분산 표현 학습
- 얻어진 학습 데이터는 인공 신경망 학습에 사용됨



- 입력값: 각 입력 단어의 원-핫 벡터
- 입력 문장 내 모든 단어의 임베딩 벡터를 평균 내어 중심 단어의 임베딩 벡터 예측
- 입력 단어는 원-핫 벡터로 표현돼 **투사층(Projection Layer)**에 입력됨
 - 투사층: 원-핫 벡터의 인덱스에 해당하는 임베딩 벡터를 반환하는 순람표 (Lookup table, LUT) 구조
- 투사층을 통과하면 각 단어는 E 크기의 임베딩 벡터로 변환, 벡터의 평균값 계산
- 계산된 평균 벡터를 가중치 행렬 W 과 곱하면 V 크기의 벡터를 얻고 이 벡터에 소프트맥스 함수를 이용해 중심 단어 예측

6.4.3 Skip-gram

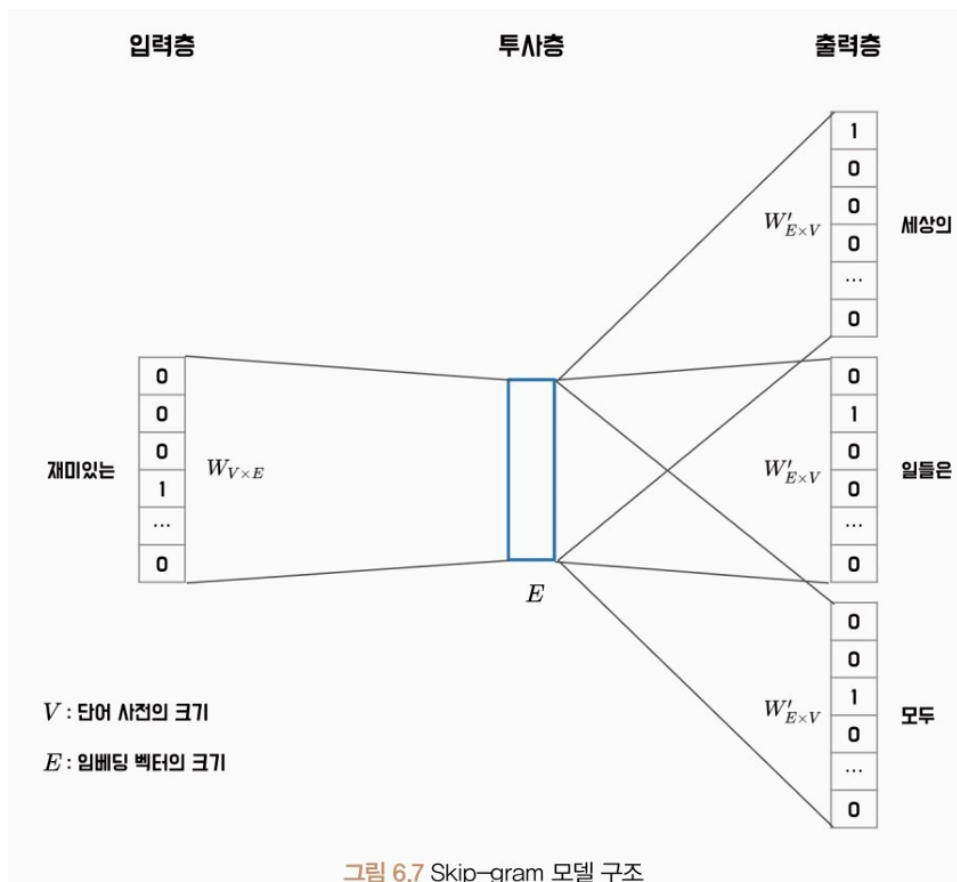
CBOW와 반대로, 중심 단어를 입력으로 받아서 주변 단어를 예측하는 모델

- 중심 단어 기준, 양쪽으로 윈도우 크기만큼의 단어들을 주변 단어로 삼아 훈련 데이터셋 생성
- 중심 단어와 각 주변 단어를 하나의 쌍으로 하여 모델 학습
- 윈도우 크기 2일 때 학습 데이터 구성

입력 문장	학습 데이터
(세상의 재미있는 일들은)모두 밤에 일어난다	(세상의 재미있는), (세상의 일들은)
(세상의 재미있는 일들은 모두)밤에 일어난다	(재미있는 세상의), (재미있는 일들은), (재미있는 모두)
(세상의 재미있는 일들은 모두 밤에)일어난다	(일들은 세상의), (일들은 재미있는), (일들은 모두), (일들은 밤에)
세상의(재미있는 일들은 모두 밤에 일어난다)	(모두 재미있는), (모두 일들은), (모두 밤에), (모두 일어난다)
세상의 재미있는(일들은 모두 밤에 일어난다)	(밤에 일들은), (밤에 모두), (밤에 일어난다)
세상의 재미있는 일들은(모두 밤에 일어난다)	(일어난다 모두), (일어난다 밤에)

그림 6.6 Skip-gram의 학습 데이터 구성

- Skip-gram과 CBoW 학습 데이터 구성 방식 차이
 - CBoW: 하나의 윈도우에서 하나의 학습 데이터 생성
 - Skip-gram: 중심 단어와 주변 단어를 하나의 쌍으로 하여 여러 학습 데이터 생성
 - ⇒ Skip-gram이 더 많은 학습 데이터 추출, 보다 뛰어난 성능, 드물게 등장하는 단어를 잘 학습, 단어 벡터 공간에서 더 유의미한 관계 형성
- Skip-gram의 모델 구조



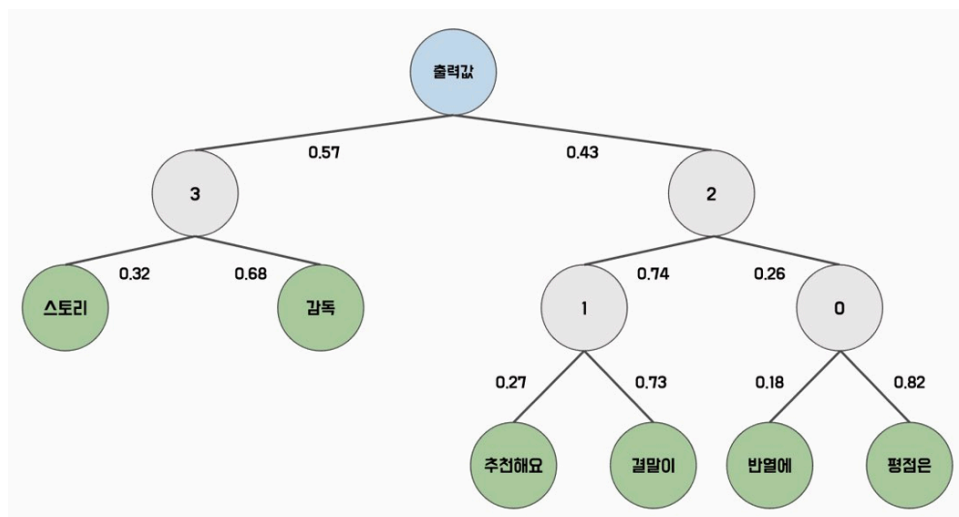
- 입력 단어의 원-핫 벡터를 투사층에 입력하여 해당 단어의 임베딩 벡터를 가져옴

- 입력 단어의 임베딩과 W 가중치와의 곱셈 $\rightarrow V$ 크기의 벡터, 이 벡터에 소프트맥스 연산을 취함으로써 주변 단어 예측
- 소프트맥스 연산: 모든 단어를 대상으로 내적 연산 수행
- 단점: 말뭉치의 크기가 커지면 필연적으로 단어 사전 크기도 커져, 학습 속도가 느려짐
 - 완화 방법: 계층적 소프트맥스, 네거티브 샘플링 기법

6.4.4 계층적 소프트맥스

출력층을 이진 트리(Binary tree) 구조로 표현해 연산 수행

- 자주 등장하는 단어일수록 트리의 상위 노드에 위치, 드물수록 하위 노드에 배치
- 일반적인 소프트맥스 연산에 비해 빠른 속도와 효율성
- 계층적 소프트맥스 구조



- 각 노드는 학습이 가능한 벡터를 가짐
- 입력값은 해당 노드의 벡터와 내적값을 계산한 후 시그모이드 함수를 통해 확률 계산
- 리프 노드는 가장 깊은 노드로, 각 단어를 의미하며, 모델은 각 노드의 벡터를 최적화하여 단어를 잘 예측할 수 있게 된다
- 각 단어의 확률은 경로 노드의 확률을 곱해서 구함
- 단어 사전의 크기를 V 라 했을 때 시간 복잡도
 - 일반 소프트 맥스 $O(V) \rightarrow$ 계층적 소프트맥스 $O(\log_2 V)$

6.4.5 네거티브 샘플링

Word2Vec 모델에서 사용되는 확률적인 샘플링 기법으로 전체 단어 집합에서 일부 단어를 샘플링하여 오답 단어로 사용

- 학습 원도 내에 등장하지 않는 단어를 n 개 추출하여 정답 단어와 함께 소프트맥스 연산 수행
- 전체 단어의 확률을 계산할 필요없이 모델을 효율적으로 수행 가능
- 추출할 단어 n 은 일반적으로 5~20개 사용

수식 6.9 네거티브 샘플링의 추출 확률

$$P(w_i) = \frac{f(w_i)^{0.75}}{\sum_{j=0}^V f(w_j)^{0.75}}$$

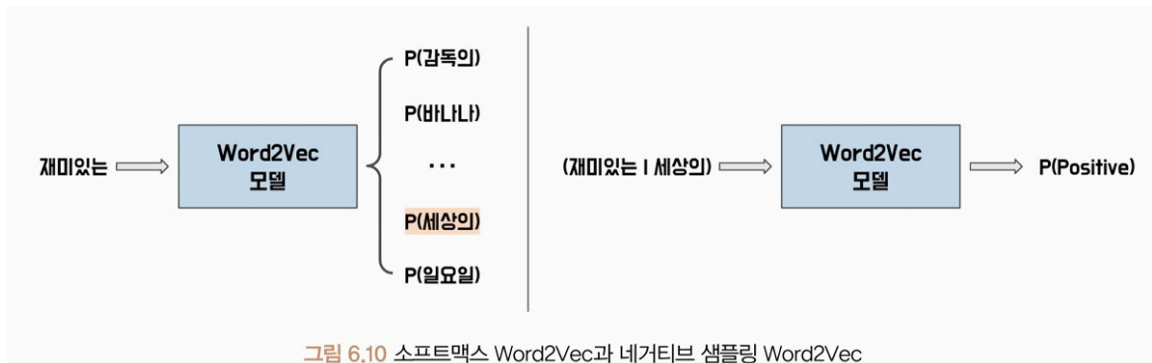
- $f(w_i)$: 각 단어 w_i 의 출현 빈도수
- $P(w_i)$: 단어 w_i 가 네거티브 샘플로 추출될 확률
- 최적의 값: 출현 빈도수에 0.75 제곱한 값을 정규화 상수로 사용
- 입력 단어 쌍이 데이터로부터 추출된 단어 쌍인지, 네거티브 샘플링으로 생성된 단어 쌍인지 이진 분류 → 이를 위해 로지스틱 회귀 모델을 사용하며, 이 모델의 학습 과정에서는 추출할 단어의 확률 분포를 구하기 위해 먼저 각 단어에 대한 가중치 학습
- 일반 Skip-gram 모델과 네거티브 샘플링 Word2Vec 모델의 훈련 데이터



- 실제 데이터에서 추출된 단어 쌍은 1로, 가짜 단어 쌍은 0으로 레이블링

⇒ 다중 분류에서 이진 분류로 학습 목적이 바뀜

- 일반적인 Word2Vec 모델과 네거티브 샘플링 Word2Vec 모델의 출력 구조 차이



- 입력 단어의 임베딩과 해당 단어가 맞는지 여부를 나타내는 레이블을 가져와 내적 연산
- 내적 연산을 통해 얻은 값은 시그모이드 함수를 통해 확률값으로 변환
- 레이블이 1인 경우 해당 확률값이 높아지도록, 레이블이 0인 경우 해당 확률값이 낮아지도록 모델의 가중치가 최적화

6.4.6 모델 실습: Skip-gram

Word2Vec 모델은 학습할 단어의 수를 V 로, 임베딩 차원을 E 로 설정해 $W_{V \times E}$ 행렬과 $W'_{E \times V}$ 행렬을 최적화 하며 학습

- $W_{V \times E}$ 행렬: **룩업(Lookup)** 연산 수행, **임베딩** 클래스 사용 시 간편하게 구현 가능
 - 임베딩 클래스는 단어나 범주형 변수와 같은 이산 변수를 연속적인 벡터 형태로 변환해 사용 가능
 - 연속적인 벡터 표현은 모델이 학습하는 동안 단어의 의미와 관련된 정보를 포착하고, 이를 기반으로 단어 간의 유사도를 계산
 - 이때 이산 변수 값을 연속적인 벡터로 변환하는 과정을 룩업이라 함
- 임베딩 클래스

```
embedding = torch.nn.Embedding(  
    num_embeddings, #이산 변수의 개수로, 단어 사전의 크기  
    embedding_dim, #임베딩 벡터 차원 수로, 벡터 크기  
    padding_idx=None, #패딩 토큰의 인덱스를 지정해 해당 인덱스의 임베딩 벡터 0으로 설정
```



```
max_norm=None, #임베딩 벡터의 최대 크기 지정
norm_type=2.0 #임베딩 벡터의 크기를 제한하는 방법 선택
)
```

#예제 6.3 기본 Skip-gram 클래스

- 계층적 소프트맥스나 네거티브 샘플링 효율적 기법X, 기본 형식의 Skip-gram
- 단순히 입력 단어와 주변 단어를 룩업 테이블에서 가져와 내적 계산 후 손실 함수를 통해 예측 오차를 최소화하는 방식으로 학습

#예제 6.4 영화 리뷰 데이터셋 전처리

- 데이터 세트: 코포라 라이브러리의 네이버 영화 리뷰 감정 분석 데이터셋
- corpus.test로 테스트 세트 불러오고 Okt 토큰나이저로 형태소 추출

#예제 6.5 단어 사전 구축

- Okt 토큰나이저를 통해 토큰화된 데이터를 활용해 build_vocab 함수로 단어 사전 구축
- n_vocab: 구축할 단어 사전 크기
 - 문서 내 n_vocab 보다 많은 종류의 토큰이 있다면, 가장 많이 등장한 토큰 순서로 사전 구축
- special_tokens: 특별한 의미를 갖는 토큰
 - <unk>: OOV에 대응하기 위한 토큰, 단어 사전 내 없는 모든 단어에 대해 대체

#예제 6.6 Skip-gram의 단어 쌍 추출

- get_word_pairs 함수: 함수 토큰을 입력 받아 전처리
- window_size: 주변 단어를 몇 개까지 고려할 것인지 설정
- idx: 현재 단어의 인덱스
- center_word: 중심 단어
- window_start, window_end: 현재 단어에서 얼마나 멀리 떨어진 주변 단어를 고려할 것인지
- 출력 결과: [중심 단어, 주변 단어] 구성

- 임베딩 층은 단어의 인덱스를 입력 받기 때문에 단어 쌍 → 인덱스 쌍 변환 필요!

#예제 6.7 인덱스 쌍 변환

- `get_index_pairs` 함수: `get_word_pairs` 함수에서 생성된 단어 쌍을 토큰 인덱스 쌍으로 변환
- `word_pairs`와 단어와 단어에 해당하는 ID를 매핑한 딕셔너리인 `token_to_id`로 인덱스 쌍 생성
- 딕셔너리의 `get` : 토큰이 단어 사전 내에 있으면 인덱스 반환, 없으면 `<unk>` 인덱스 반환

#예제 6.8 데이터로더 적용

- `index_pairs`: `get_index_pairs` 함수에서 생성된 중심 단어와 주변 단어 토큰의 인덱스 쌍 리스트
- 위의 리스트를 텐서로 변환 → $[N, 2]$ 구조, 중심 단어와 주변 단어로 나눠 데이터셋으로 변환

#예제 6.9 Skip-gram 모델 준비 작업

- `VanillaSkipgram`: 단어 사전 크기에 전체 단어 집합의 크기를 전달하고 임베딩 크기는 128
- 손실 함수: 단어 사전 크기만큼 클래스가 있는 분류 문제 → 교차 엔트로피 사용
 - 교차 엔트로피: 내부적으로 소프트맥스 연산 수행, 신경망 출력값을 후처리X 사용

#예제 6.10 모델 학습

- 모델 학습이 완료되면 행렬 하나를 선택해 임베딩 값 추출

#예제 6.11 임베딩 값 추출

- 임베딩 값으로 단어 간 유사도 확인 가능 → 코사인 유사도가 가장 일반적
- 코사인 유사도: 두 벡터 간의 각도를 이용하여 유사도 계산, 유사할수록 1에 가까움
 - 두 벡터의 내적을 벡터의 크기(유클리드 노름)의 곱으로 나누어 계산

수식 6.10 코사인 유사도

$$\text{cosine similarity}(a, b) = \frac{a \cdot b}{\|a\| \|b\|}$$

- a, b: 유사도를 계산하려는 벡터
- 두 벡터를 내적한 벡터의 크기의 곱을 나눠 계산

#예제 6.12 단어 임베딩 유사도 계산

- cosine_similarity 함수: 입력 단어와 단어 사전 내의 모든 단어와의 코사인 유사도 계산
- a는 임베딩 토큰, b는 임베딩 행렬을 의미
- top_n_index 함수: 유사도 행렬을 내림차순으로 정렬, 어떤 단어가 가장 가까운 단어인지 반환

6.4.7 모델 실습: Gensim

젠심(Gensim)

Word2Vec과 같은 자연어 처리 모델을 쉽게 구성할 수 있다.

- 대용량 텍스트 데이터의 처리를 위한 메모리 효율적인 방법 제공
- 학습된 모델 저장 관리, 유사도 관련 기능 → 자연어 처리 관련 다양한 기능 제공
- 젠심 라이브러리 설치

```
!pip install gensim
```

- 젠심 라이브러리의 Word2Vec 클래스

```
word2vec = gensim.models.Word2Vec(  
    sentences=None, #모델의 학습 데이터를 나타내며 토큰 리스트로 표현  
    corpus_file=None, #학습 데이터를 파일로 입력할 때 파일의 경로  
    vector_size=100, #학습할 임베딩 벡터 크기  
    alpha=0.025, #모델의 학습률  
    window=5, #학습 데이터를 생성할 윈도우 크기  
    min_count=5, #학습에 사용할 단어의 최소 빈도
```

```

workers=3, #빠른 학습을 위해 병렬 학습할 스레드의 수
sg=0, #Skip-gram 모델 사용 여부 설정
hs=0, #계층적 소프트맥스 사용 여부
cbow_mean=1, #CBoW 모델로 구성할 때 사용되는 하이퍼파라미터로 중심
단어
negative=5, #네거티브 샘플링 단어의 수 의미
ns_exponent=0.75, #네거티브 샘플링 확률의 지수
max_final_vocab=None, #단어 사전의 최대 크기
epochs=5, #학습 에폭 수
batch_words=10000 #몇 개의 단어로 학습 배치를 구성할지 결정
)

```

#예제 6.13 Word2Vec 모델 학습

- 모델 학습 속도는 앞선 클래스와 비교했을 때 매우 빠름
- word2vec 인스턴스는 파이토치의 저장 메서드와 동일한 방법으로 저장

#예제 6.14 임베딩 추출 및 유사도 계산

- wv속성: 학습된 단어 벡터 모델을 포함한 Word2VecKeyedVectors 객체 반환
 - ↳ 이 객체는 단어 벡터 검색과 유사도 계산 등 작업 수행에 사용
 - 가장 유사한 단어를 찾는 메서드, 두 단어 간의 유사도 계산 메서드 제공

Word2Vec의 분포가설 장단점

- 장점: 쉽고 빠른 단어의 임베딩 학습
- 단점: 단어의 형태학적 특징을 반영하지 못한다는 한계

6.5 fastText

2015년 메타의 FAIR 연구소에서 개발한 오픈소스 임베딩 모델로, 텍스트 분류 및 텍스트마 이닝을 위한 알고리즘

- 단어와 문장을 벡터로 변환하는 기술을 기반

- 벡터화 기술은 Word2Vec과 유사하나, 단어의 하위 단어를 고려하므로 더 높은 정확도와 성능
 - N-gram을 사용해 단어를 분해하고 벡터화하는 방법으로 동작
- 단어의 벡터화를 위해 <, >와 같은 특수 기호 사용
- 기호가 추가된 단어는 N-gram을 사용하여 하위 단어 집합으로 분해
 - 분해된 하위 단어 집합에는 나뉘지지 않은 단어 자신도 포함
 - 단어 집합이 만들어지면 각 하위 단어는 고유한 벡터값을 가짐



- '서울특별시' 토큰에 대한 단계

1. 토큰의 양 끝에 '<'와 '>'를 붙여 토큰의 시작과 끝을 인식할 수 있게 한다.
2. 분해된 토큰은 N-gram을 사용하여 하위 단어 집합으로 분해한다. 트라이그램을 사용한다면, '서울특별시'는 [<서울, 서울특, 울특별, 특별시, 별시>]로 분해된다.
3. 분해된 하위 단어 집합에는 나뉘지지 않은 토큰 자체도 포함한다. 이렇게 하위 단어 집합이 만들어지면, 각 하위 단어는 고유한 벡터값을 갖는다.

6.5.1 모델 실습

- 두 단어를 고정 길이 벡터 형태로 표현하기 위해 분산 표현 학습을 수행, 주변 단어의 정보를 활용하여 단어의 의미 파악
- 단어 간 유사도 측정, 비슷한 의미를 가진 단어를 유사한 벡터로 표현 → 공간상 가깝게 임베딩
- 네거티브 샘플링 기법을 사용해 학습
- FastText 클래스

```

fasttext = gensim.models.FastText(
    sentences=None,
    corpus_file=None,
    vector_size=100,
    alpha=0.025,
    window=5,
    min_count=5,
    workers=3,
    sg=0,
    hs=0,
    cbow_mean=1,
    negative=5,
    ns_exponent=0.75,
    max_final_vocab=None,
    epochs=5,
    batch_words=10000,
    min_n = 3, #N-gram 최솟값
    max_n=6 #N-gram최댓값
)

```

#예제 6.15 KorNLI 데이터세트 전처리

- KorNLI 데이터세트: 한국어 자연어 추론을 위한 데이터세트
 - 자연어 추론: 두 개 이상의 문장이 주어졌을 때, 두 문장 간의 관계를 분류하는 작업
- 주어진 문장이 함의 관계, 중립관계, 불일치 관계 중 어느 관계인지 분류 가능
- get_all_texts 메서드: 모든 문장을 튜플 형태로 반환
- get_all_pairs 메서드: 입력 문장과 쌍을 이루는 대응 문장을 튜플 형태로 반환
- 입력 단어의 구조적 특징 학습 가능 → 띄어쓰기 수준으로 단어 토큰화하여 학습 진행

#예제 6.16 fastText 모델 실습

- Word2Vec과 다르게 OOV 단어를 대상으로도 의미 있는 임베딩 추출 가능

#예제 6.17 fastText OOV 처리

- 단어 사전에 없는 단어의 임베딩 추출 방법, 유사도 계산 방법

- `wv.index_to_key` 메서드: 학습된 단어 사전을 나타내는 리스트
- OOV 문제 효과적으로 해결 → 특히 한국어같은 언어는 형태적 구조를 갖고 있어, 하위 단어로 나누어 임베딩을 학습하는 fastText 접근 방식을 통해 OOV 문제 해결

6.6 순환 신경망

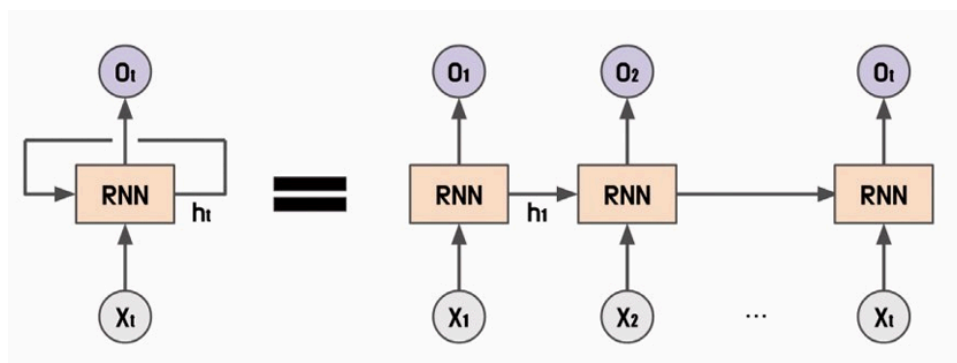
순서가 있는 **연속적인 데이터** 처리하는 데 적합한 구조를 갖고 있다.

- 각 시점의 데이터가 이전 시점의 데이터와 독립적이지 않다는 특성 때문에 효과적으로 작동
- 자연어는 한 단어가 이전의 단어들과 상호작용하여 문맥을 이루고 의미 형성
→ 자연어는 연속형 데이터 특성을 가짐
- 긴 문장일수록 앞선 단어들과 뒤따르는 단어들 사이에 강한 상관관계 존재

6.6.1 순환 신경망

연속적인 데이터를 처리하기 위해 개발된 인공 신경망의 한 종류

- 이전에 처리한 데이터를 다음 단계에 활용하고 현재 입력 데이터와 함께 모델 내부에서 과거의 상태를 기억해 현재 상태를 예측하는 데 사용
- 시계열 데이터, 자연어 처리, 음성 인식 및 기타 시퀀스 데이터와 같은 도메인에서 사용
 - 일반적으로 길이가 가변적, 순서에 따른 의미 존재
- 연속형 데이터를 순서대로 입력받아 처리하며 각 시점마다 은닉상태의 형태로 저장
- 셀: 각 시점의 데이터를 입력으로 받아 은닉 상태와 출력값을 계산하는 노드
- 순환 신경망 셀



- 순환 신경망의 은닉 상태

$$h_t = \sigma_h(h_{t-1}, x_t)$$

$$h_t = \sigma_h(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$

- 시그마: 순환 신경망의 은닉 상태를 계산하기 위한 활성화 함수
- 시그마는 가중치와 편향을 이용해 계산
- 순환 신경망의 출력값

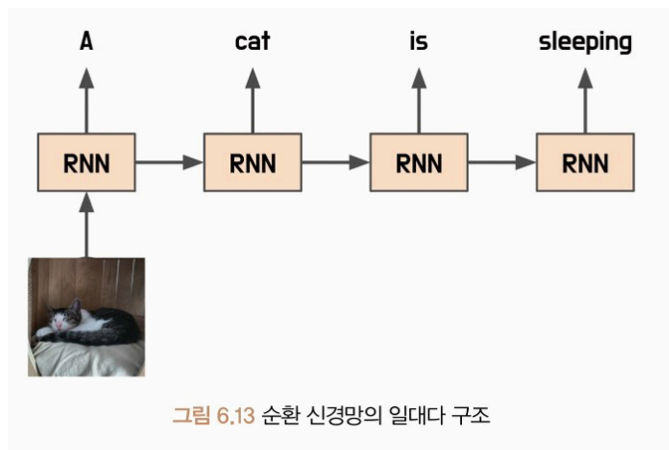
$$y_t = \sigma_y(h_t)$$

$$y_t = \sigma_y(W_{hy}h_t + b_y)$$

- 시그마: 순환 신경망의 출력값을 계산하기 위한 활성화 함수
- 출력값 계산 방법도 가중치와 편향을 이용해 계산
- 순환 신경망의 출력값은 이전 시점의 정보를 현재 시점에서 활용해 입력 시퀀스의 패턴을 파악하고 출력값을 예측하므로 연속형 데이터 처리

순환 신경망은 다양한 구조로 모델 설계 가능하다

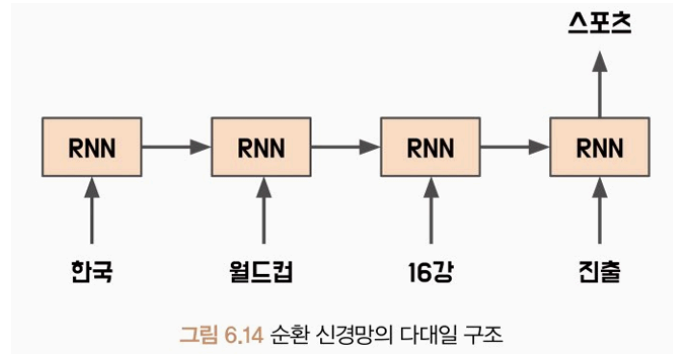
일대다 구조: 하나의 입력 시퀀스에 대해 여러 개의 출력값을 생성하는 순환 신경망 구조



- 이미지 데이터를 입력으로 받으면 이미지에 대한 설명을 출력하는 이미지 캡셔닝 모델
- 입력 시퀀스는 이미지, 출력값은 이미지에 대한 설명문장들로 구성

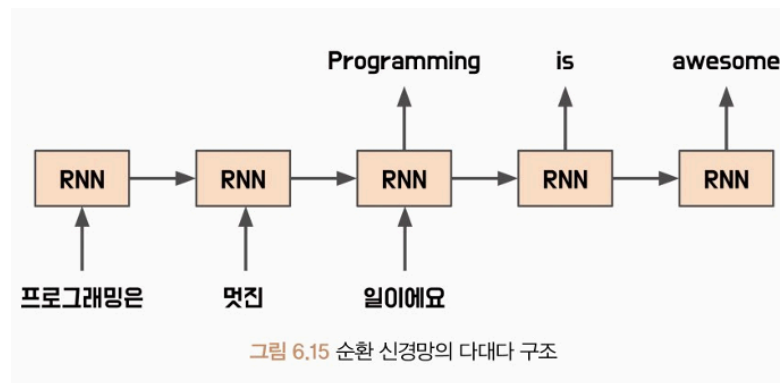
- 출력 시퀀스의 길이를 미리 알고 있어야 함

다대일 구조 : 여러 개의 입력 시퀀스에 대해 하나의 출력값을 생성하는 순환 신경망 구조



- 감성 분류 분야에서 특정 문장의 감성을 예측하는 작업 수행 가능
- 문장 분류, 자연어 추론 등에 적용 가능

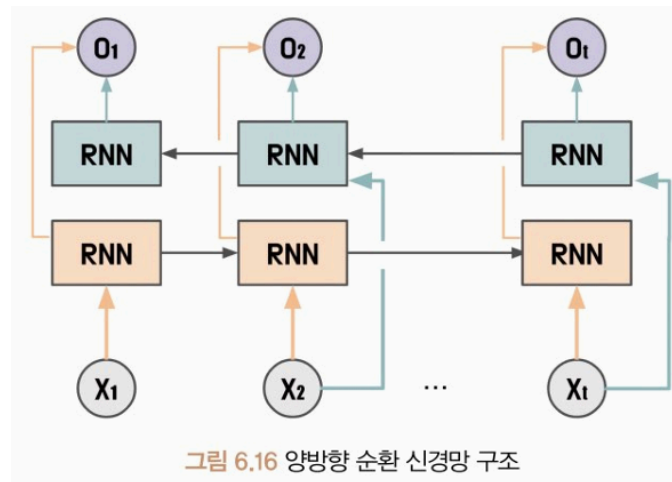
다대다 구조 : 입력 시퀀스와 출력 시퀀스의 길이가 여러 개인 경우에 사용되는 순환 신경망 구조



- 다양한 분야에서 활용됨
- 입력 시퀀스와 출력 시퀀스의 길이 다른 경우 → 패딩 추가 or 잘라내기
- 다대다 구조: 시퀀스-시퀀스 구조
 - 인코더: 입력 시퀀스를 처리해 고정 크기의 벡터 출력
 - 디코더: 이 벡터를 입력으로 받아 출력 시퀀스 생성

양방향 순환 신경망

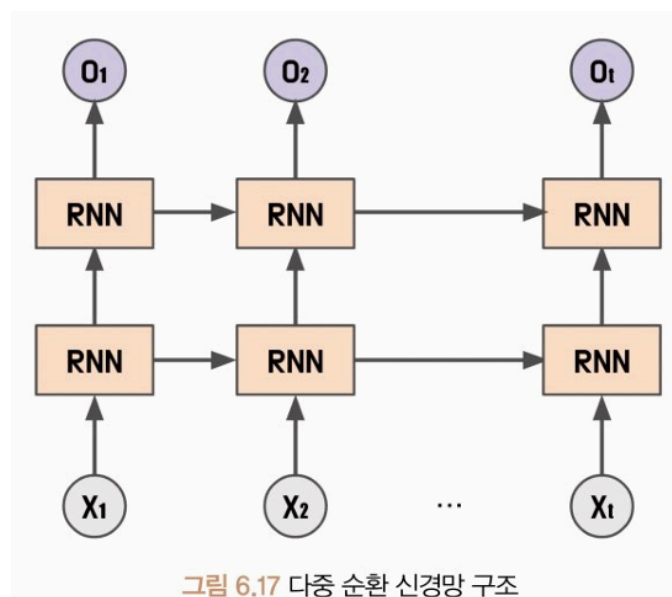
기본적인 순환 신경망에서 시간 방향을 양방향으로 처리할 수 있도록 고안된 방식



- 이전 시점의 은닉 상태뿐만 아니라 이후 시점의 은닉 상태도 함께 이용
- t 시점 이후의 데이터도 t 시점의 데이터를 예측하는 데 사용
- 입력 데이터를 순방향으로 처리하는 것만 아니라 역방향으로도

다중 순환 신경망

여러 개의 순환 신경망을 연결하여 구성한 모델로 각 순환 신경망이 서로 다른 정보를 처리하도록 설계되어 있다



- 여러 개의 순환 신경망 층으로 구성, 각 층에서의 출력값은 다음 층으로 전달 돼 처리
- 데이터의 다양한 특징 추출 가능 → 성능 향상

- 층이 깊어질수록 더 복잡한 패턴 학습 가능
- 단점: 학습시간이 오래 걸리고, 기울기 소실 문제 발생 가능성이 큰

순환 신경망 클래스

일대일 구조 순환 신경망부터 다중 순환 신경망까지 쉽게 구현 가능

```
rnn = torch.nn.RNN(
    input_size,
    hidden_size,
    num_layers=1, #순환 신경망의 층수
    nonlinearity = 'tanh', #활성화 함수
    bias = False, #편향 값 사용유무 설정
    batch_first=True, #입력 배치 크기를 첫 번째 차원으로 사용할지 여부 결정
    dropout=0, #과대적합 방지를 위한 드롭아웃 확률 설정
    bidirectional=False #순환신경망 구조가 양방향으로 처리할지 여부 결정
)
```

#예제 6.18 양방향 다층 신경망

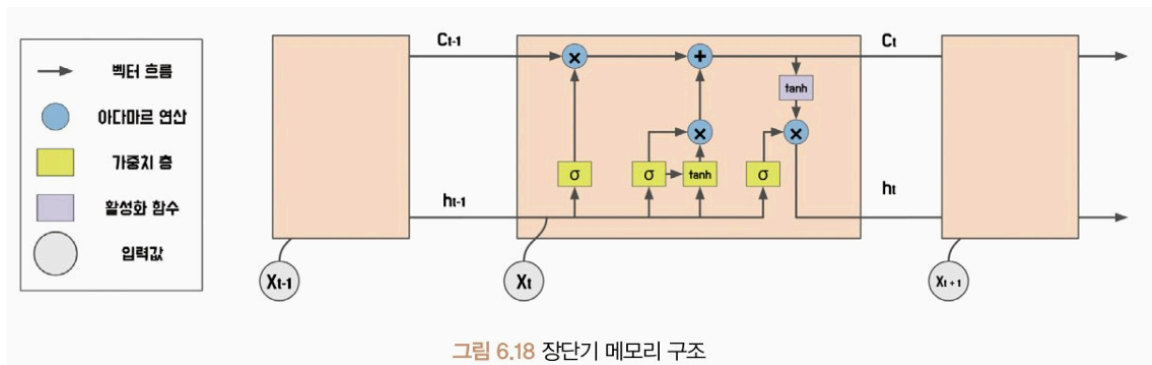
- 순전파 연산 시, 입력값과 초기 은닉 상태로 순방향 연산을 수행해 출력값과 최종 은닉상태 반환
- 입력값 차원: [배치 크기, 시퀀스 길이, 입력 특성 크기]로 전달
- 초기 은닉 상태: [계층 수 * 양방향 여부 +1. 배치크기, 은닉 상태 크기]로 구성
- 출력값: [배치 크기, 시퀀스 길이, (양방향 여부+1) * 은닉 상태 크기]
- 최종 은닉 상태는 초기 은닉 상태와 동일한 차원으로 반환

6.6.2 장단기 메모리

기존 순환 신경망이 갖고 있던 기억력 부족과 기울기 소실 문제를 해결한 모델

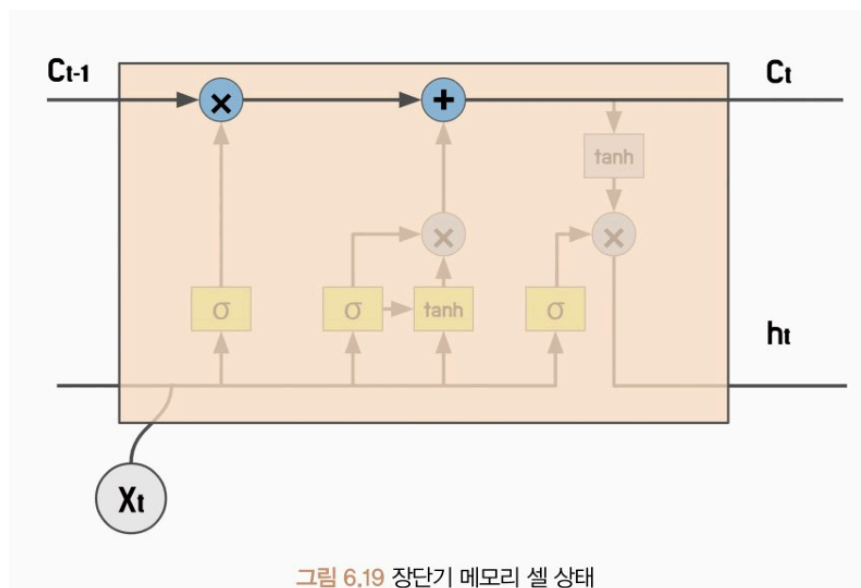
- 순환 신경망의 단점: 앞선 시점에서의 정보를 끊임없이 반영하기에 학습 데이터의 크기가 커질수록 앞서 학습한 정보가 충분히 전달 X
 - ⇒ 장기 의존성 문제 발생 가능성, 기울기 소실/폭주 발생 가능성
 - ⇒ 이 문제를 해결하기 위해 장단기 메모리 사용

- 장단기 메모리: 순환 신경망과 비슷한 구조를 가지지만, **메모리 셀**과 **게이트** 구조를 도입해 장기 의존성 문제와 기울기 소실 문제 해결



- 장단기 메모리는 **셀 상태**, **망각 게이트**, **기억 게이트**, **출력 게이트**로 정보의 흐름 제어
 - 셀 상태: 정보를 저장하고 유지하는 메모리 역할을 하며 출력 게이트와 망각 게이트에 의해 제어 됨
 - 망각 게이트: 장단기 메모리에서 이전 셀 상태에서 어떤 정보를 삭제할지 결정하는 역할
 - 입력 게이트: 새로운 정보를 어떤 부분에 추가할지 결정하는 역할
 - 출력 게이트: 셀 상태의 정보 중 어떤 부분을 출력할지 결정하는 역할

장단기 메모리 구조



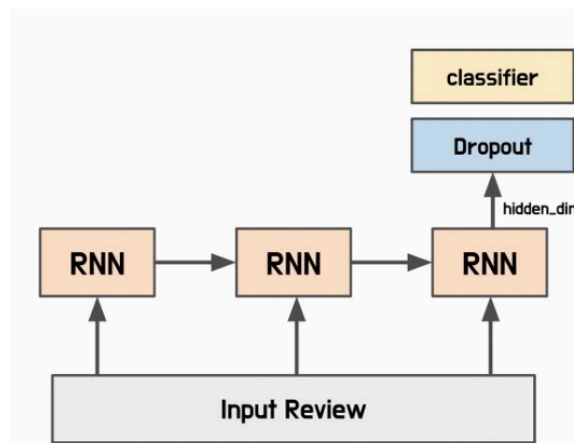
#예제 6.19 양방향 다층 장단기 메모리

- 장단기 메모리 클래스는 순전파 연산 시, 입력값과 초기 은닉 상태, 초기 메모리 셀 상태로 순방향 연산을 수행해 출력값과 최종 은닉 상태, 최종 메모리 셀 상태를 반환
- 선형 투사를 통해 출력층의 차원을 변경할 수 있으므로 투사 크기가 0보다 큰 경우 출력층의 차원을 투사 크기로 입력
- 선형 투사가 적용됐으므로 출력값은 [배치 크기, 시퀀스 길이, (양방향 여부 + 1) * 투사 크기(또는 은닉 상태 크기)] 로 구성

6.6.3 모델 실습

순환 신경망과 장단기 메모리를 활용해 문장 긍정/부정 분류 모델 학습

- 긍정/부정 분류 모델의 구조



#예제 6.20 문장 분류 모델

- 임베딩 층을 구성할 때 사용하는 단어 사전 크기와 순환 신경망 클래스와 장단기 메모리 클래스에서 사용하는 매개변수를 입력으로 전달받는다
- 모델 종류: 순환 신경망? 장단기 메모리 여부?
- 순방향 메서드에서는 입력 받은 정수 인코딩을 임베딩 계층에 통과시켜 임베딩 값을 얻음
- 출력값의 마지막 시점만 활용할 예정이므로 마지막 시점의 결괏값만 분리해 분류기 계층에 전달

#예제 6.21 데이터셋 불러오기

- 데이터셋: 코포라 라이브러리의 네이버 영화 리뷰 감정 분석
- corpus.test: 테스트 세트, 이 셋을 학습 데이터와 테스트 데이터로 분리

#예제 6.22 데이터 토큰화 및 단어 사전 구축

- Okt 토큰라이저를 사용해 데이터셋을 토큰화하고 단어 사전 구축
- build_vocab 함수는 예제 6.5와 동일 but 문장 길이를 맞추기 위해 <pad> 토큰을 special_tokens에 추가

#예제 6.23 정수 인코딩 및 패딩

- 학습 데이터셋과 테스트 데이터셋을 단어 사전에 있는 정수로 인코딩
- 정수로 인코딩 → 모델이 텍스트 데이터 처리가 쉬움
- 컴퓨터는 텍스트를 처리하는 것보다 숫자를 처리하는 것이 훨씬 빠르고 효율적
- 최대길이: 입력 텍스트의 길이를 고려해 결정
- pad_sequences 함수: 너무 긴 문장은 최대 길이로 줄이고, 너무 작은 길이라면 최대 길이와 동일한 크기로 변환
- OOV의 경우 1(<unk>)로 인코딩, 시퀀스 길이가 짧으면 최대 길이(32)가 될 수 있게 0(<pad>) 토큰이 추가

#예제 6.24 데이터로더 적용

- 텐서 데이터셋 클래스는 파이토치 텐서 형태를 입력값으로 받음
⇒ 정수 인코딩과 라벨값을 파이토치 텐서 형태로 변환

#예제 6.25 손실 함수와 최적화 함수 정의

- 학습 적용 파라미터 선언(은닉 상태 크기, 임베딩 벡터 크기)
- BCEWithLogisticLoss ⇒ BCELoss클래스+Sigmoid클래스
- 최적화 함수: RMSProp 적용
 - 모든 기울기 누적X, 지수 가중 이동 평균을 사용해 학습률 조절
- 기울기 제공값의 평균값이 작아지면 학습률 증가

#예제 6.26 모델 학습 및 테스트

- 에폭마다 검증 손실, 검증 정확도 확인
- 손실이 감소, 정확도 상승

#예제 6.27 학습된 모델로부터 임베딩 추출

- 모델 학습 과정에서 임베딩 계층을 비롯한 순환 신경망 내 여러 가중치 최적화
- 학습된 임베딩 계층의 가중치를 동일한 단어 사전을 사용하는 토큰의 임베딩 값으로 사용 가능

#예제 6.28 사전 학습된 모델로 임베딩 계층 초기화

- 사전 학습된 모델로 임베딩 계층을 초기화하는 경우 넘파이 배열로 초기화 가능. 단 이때 <pad> 토큰과 <unk> 토큰은 초기화에서 제외
- 임베딩 계층은 from_pretrained 메서드로 초기화 가능
 - 클래스의 self.embedding 값으로 적용

#예제 6.29 사전 학습된 임베딩 계층 적용

- 사전 학습된 임베딩 매개변수가 None이 아니라면 전달된 값을 임베딩 계층으로 초기화

#예제 6.30 사전 학습된 임베딩을 사용한 모델 학습

- 사전 학습된 임베딩 사용은 모델 성능 개선 방법 중 하나이지만, 학습 데이터의 양이 충분히 많다면, 모델의 목적에 맞게 새로운 임베딩 층을 학습하는 것이 더 좋은 결과를 얻을 수 있음
- 사전 학습된 임베딩을 사용하더라도 해당 언어와 문제에 맞는 임베딩 선택이 중요!

6.7 합성곱 신경망

주로 이미지 인식과 같은 컴퓨터비전 분야의 데이터를 분석하기 위해 사용되는 인공신경망 종류

- 입력 데이터의 지역적 특징을 추출하는데 특화된 구조를 갖고 있으며 합성곱 연산 사용

- 특정 영역 안에서 연산 수행 가능 → 지역 특징 효과적 추출
- 여러 영역의 지역 특징을 조합해 입력 데이터의 전반적인 전역 특징 파악 가능
- 연산 순서의 제약으로 병렬 처리가 어렵다는 단점
- 입력 데이터 길이가 길수록 처리 속도가 느림
- 많은 수의 가중치를 사용하면 학습이 어려워짐
- 그러나 사전 학습된 임베딩 + 합성곱 신경망 → 병렬 처리 가능, 가중치 수 감소

6.7.1 합성곱 계층

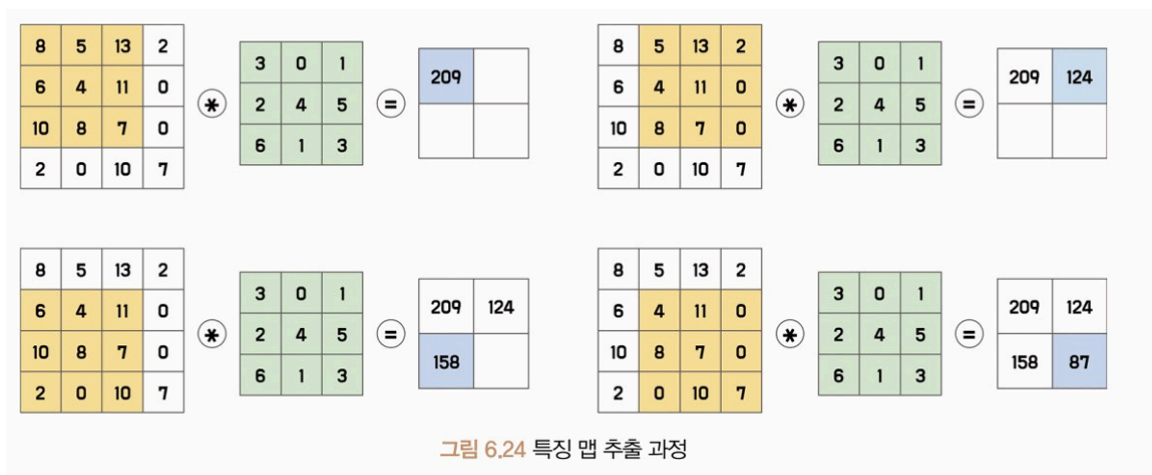
입력 데이터와 필터를 합성곱해 출력 데이터를 생성하는 계층이다

- 이미지나 음성 데이터와 같은 고차원 데이터를 처리하는 데 주로 사용
- 모델 매개변수 공유 → 과적합 방지

필터

합성곱 계층은 입력 데이터에 필터를 이용해 합성곱 연산을 수행하는 계층이다

- 커널, 윈도우라고도 불림
- 작은 크기의 정방향으로 구성
- 필터를 일정 간격으로 이동하며 입력 데이터와 합성곱 연산을 수행해 특징 맵 생성



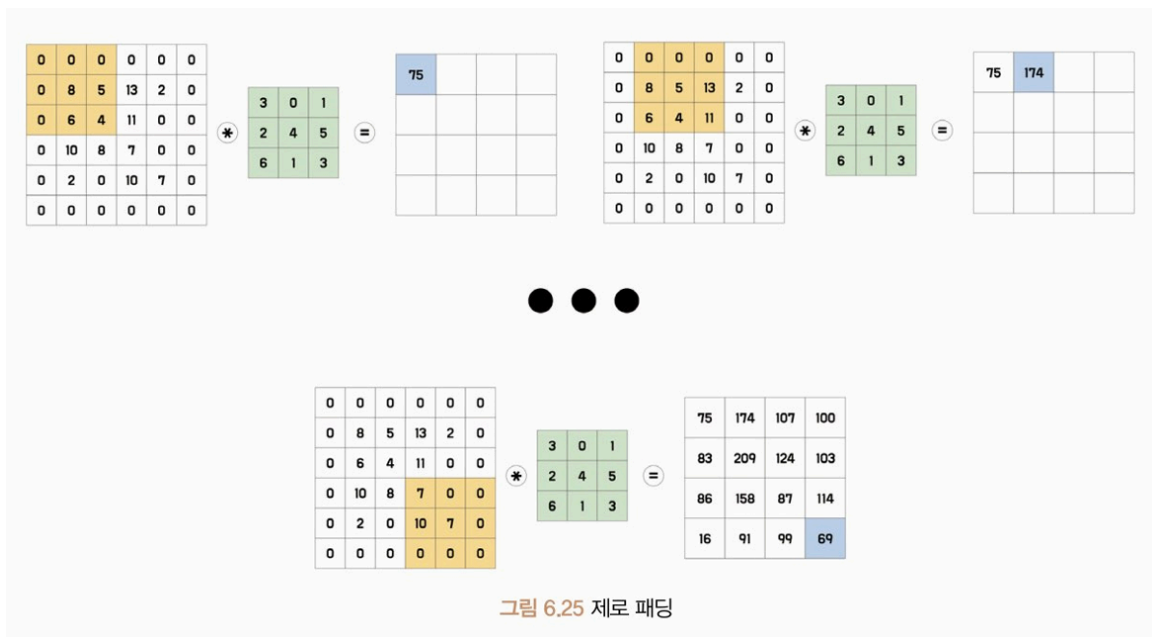
- 입력 이미지에서 필터와 대응하는 부분을 곱한 후, 모두 더한 값을 특징 맵의 한 원소로 사용

- 입력 이미지 왼쪽 상단부터 오른쪽 하단으로 이동하며 합성곱 연산 반복

패딩

가장자리 부분의 정보가 학습하는 데 제한이 있는 현상을 방지하기 위해 입력 이미지나 입력으로 사용되는 특징 맵 가장자리에 특정 값을 덧붙이는 패딩을 추가한다.

- 제로 패딩: 가장자리에 덧붙이는 패딩 값은 0으로 할당



- 입력 데이터의 크기 조정, 작은 크기의 필터로도 이미지 전체 정보 반영 가능

간격

필터가 한 번에 움직이는 크기를 의미

- 간격 크기 1: 필터가 한 픽셀씩 이동하며 합성곱 연산 수행
- 간격 크기를 조절함으로써 출력 데이터 크기 조절 가능, 입력 데이터의 공간 정보 유지/감소
 - 입력 데이터 공간 정보: 픽셀 간 상대적인 위치나 거리에 대한 정보를 의미

채널

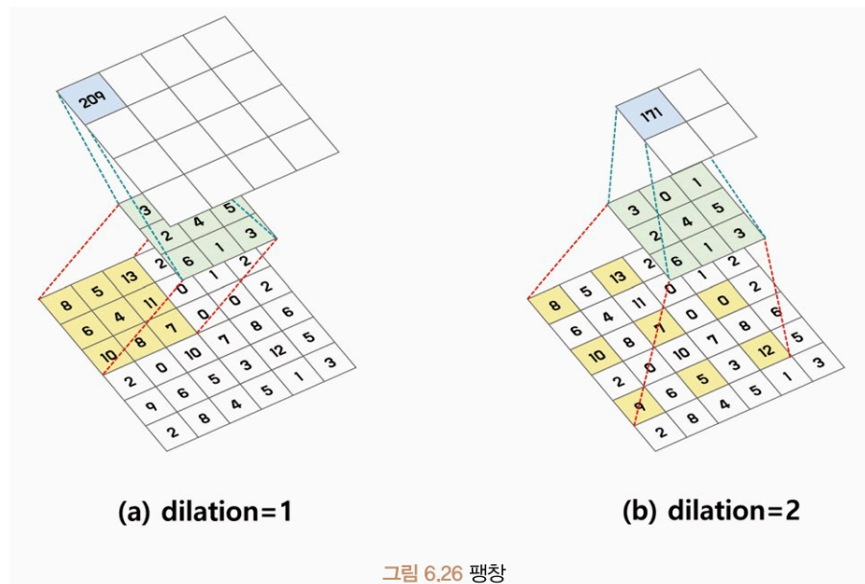
입력 데이터와 필터가 3차원으로 구성되어있을 때 같은 위치 값끼리 연산

- 입력 데이터와 필터 간의 연산은 채널에 의해 수행
- 입력 데이터의 공간 정보 유지, 추출되는 특징 확장 가능
- 채널의 개수가 많아질수록 학습할 수 있는 특징의 다양성이 증가해 모델의 표현력 증가
- 출력 채널이 많을 수록 학습 시간, 메모리 사용량 증가 → 과적합 위험

팽창

합성곱 연산을 수행할 때 입력 데이터에 더 넓은 범위의 영역을 고려할 수 있게 하는 기법

- 필터와 입력 데이터 사이에 간격을 두는 방법
- 일반적으로 팽창값은 1로 사용해 간격을 한 칸만 띄워 사용



- 팽창을 적절히 사용하면 메모 사용량 감소 효과

합성곱 계층 클래스

```
conv = torch.nn.Conv2d(
    in_channels,
    out_channels,
    kernel_size, #합성곱 연산의 필터 크기
    stride=1,
    padding=0,
```

```

dilation=1,
groups=1, #입력 채널과 출력 채널을 하나의 그룹으로 묶는 것
bias=True, #계층에 편향 값 포함 여부 설정
padding_mode="zeros" #패딩 영역에 할당되는 값 설정
)

```

수식 6.18 합성곱 계층 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - dilation \times (kernel_size - 1) - 1}{stride} + 1 \right\rfloor$$

수식 6.19 그림 6.26의 (a) 출력 크기 계산식

$$L_{in} = 6, kernel_size = 3, stride = 1, padding = 0, dilation = 1$$

$$L_{out} = \left\lfloor \frac{6 + 2 \times 0 - 1 \times (3 - 1) - 1}{1} + 1 \right\rfloor = 4$$

6.7.2 활성화 맵

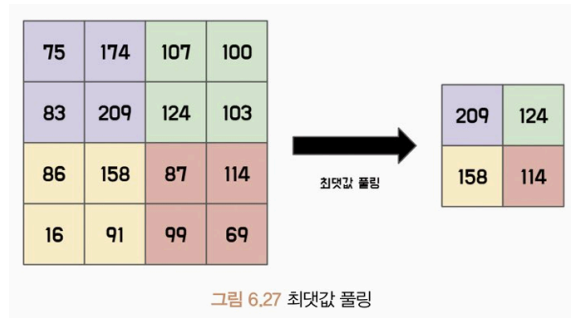
합성곱 계층의 특징 맵에 활성화 함수를 적용해 얻어진 출력값을 의미

- 특징 맵을 추출하면 이 값에 비선형성을 추가하기 위해 활성화 함수 적용
- 다음 계층의 입력값이 됨
- 합성곱 연산 + 활성화 함수 → 입력 이미지에서 추출된 추상적 특징 학습 가능

6.7.3 풀링

특징 맵의 크기를 줄이는 연산으로 합성곱 계층 다음에 적용된다

- 특징 맵의 크기를 줄여 연산량을 감소, 입력 데이터 정보 압축 효과
- 합성곱 연산과 비슷하게 필터와 간격 이용
- 최대값 풀링: 특정 크기의 필터 내 원소값 중 가장 큰 값을 선택해 특징 맵 크기 감소



- 평균값 풀링: 필터 내 원소값의 평균값으로 특징 맵 크기 감소

풀링 클래스

#2차원 최댓값 풀링 클래스

```
pool = torch.nn.MaxPool2d(
    kernel_size,
    stride=None,
    padding=0,
    dilation=1
)
```

#2차원 평균값 풀링 클래스

```
pool = torch.nn.AvgPool2d(
    kernel_size,
    stride=None,
    padding=0,
    count_include_pad=True
)
```

- 평균값 풀링 → 팅창 지원 X
- 패딩 포함: 패딩 영역의 값을 평균 계산에 포함할지 여부 설정

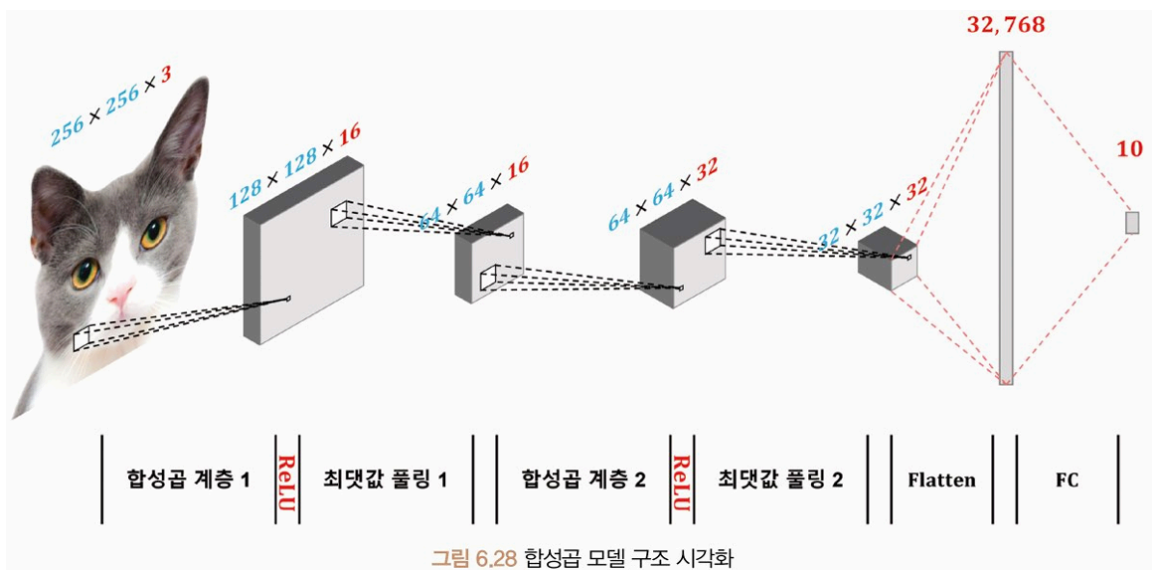
수식 6.20 평균값 풀링 출력 크기

$$L_{out} = \left\lfloor \frac{L_{in} + 2 \times padding - kernel_size}{stride} + 1 \right\rfloor$$

6.7.4 완전 연결 계층

각 입력 노드가 모든 출력 노드와 연결된 상태를 의미

- 입력과 출력 간의 모든 가능한 관계 학습 가능
- 출력 노드 수 조절 가능 → 모델 복잡성, 용량 조절에 용이
- 특징맵을 입력으로 받음 → 3차원 특징 맵은 평탄화를 통해 1차원 벡터로 변경되고 완전 연결 계층의 가중치와 내적 연산을 수행해 출력값 계산
- 모든 입력은 독립적으로 처리해 계산
- 최종적인 분류 작업 수행(소프트맥스, 시그모이드 등)



- 입력 이미지 → 합성곱 계층, ReLU, 최댓값 풀링 2번 반복, 3차원 배열 → 1차원 flatten
- 1차원 벡터를 완전 연결 계층으로 전달해 10개의 출력 벡터로 반환

#예제 6.31 합성곱 모델

- 완전 연결 계층은 선형 변환으로 구성, 입력 벡터와 가중치의 곱으로 계산
- 입력 데이터 차원 크기는 마지막 특징 맵의 높이*너비*채널로 계산

6.7.5 모델 실습

- 텍스트 데이터에서도 이미지 데이터와 같이 2차원 합성곱 필터를 사용하면 텍스트의 정보를 제대로 학습 X

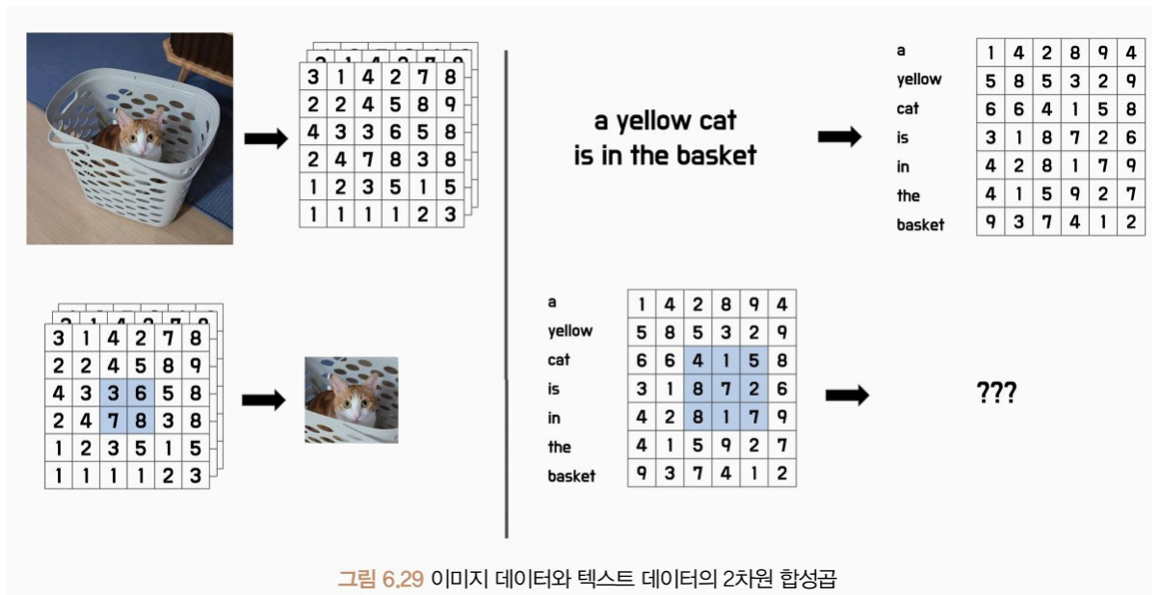


그림 6.29 이미지 데이터와 텍스트 데이터의 2차원 합성곱

⇒ 1차원 합성곱을 적용해야함!

- 텍스트 임베딩 입력값에 크기 3인 1차원 합성곱 연산을 1씩 이동하며 수행 과정

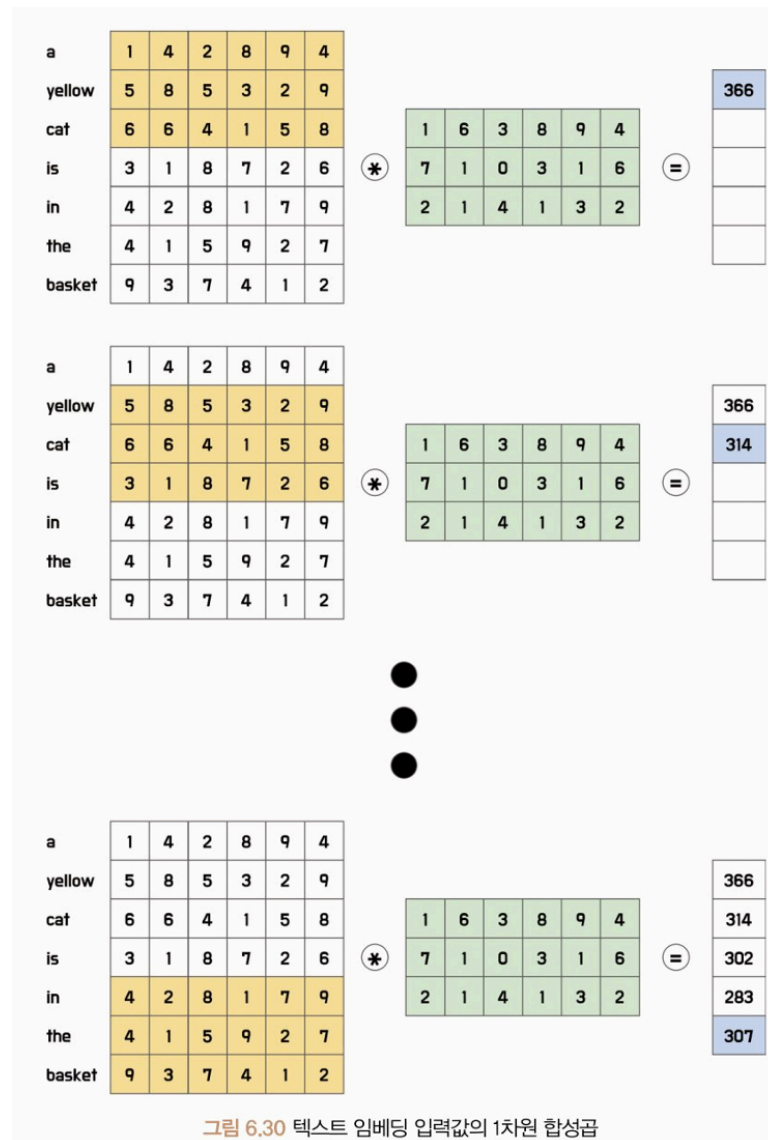
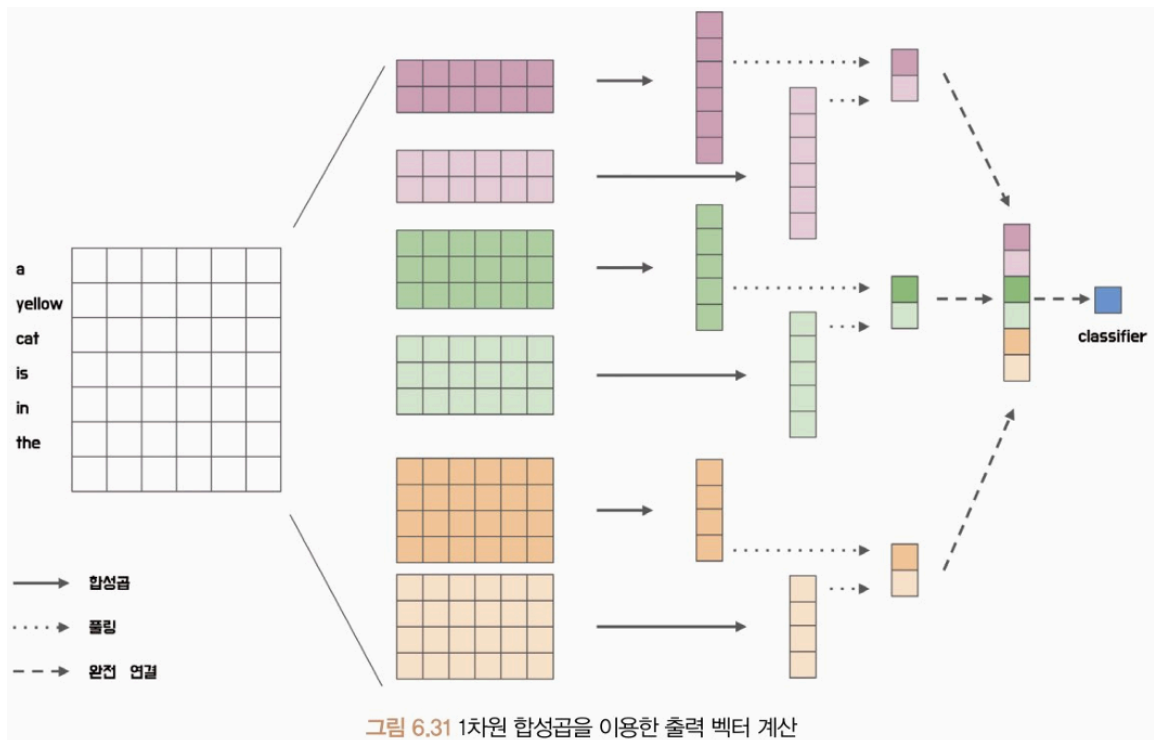


그림 6.30 텍스트 임베딩 입력값의 1차원 합성곱

- 텍스트 순서에 따른 정보 학습 가능
- 여러 개의 합성곱 필터를 이용하면 여러 개 스칼라 값 도출
- 다양한 크기의 출력 벡터를 하나로 모아 하나의 특징 벡터 생성 가능
- 크기가 각각 2, 3, 4인 필터를 2개씩 사용해서 출력 특징 벡터 얻는 과정



#예제 6.32 합성곱 기반 문장 분류 모델 정의

- 사전 학습된 임베딩 벡터, 필터 크기 리스트, 최대길이를 입력 받아 모델 구성
- 각 시퀀스는 1차원 합성곱 계층, ReLU, 최댓값 풀링으로 구성
- ModuleList: 여러 개의 서브 모듈을 리스트 형태로 저장하는 기능 제공
- permute: 차원 변경
- cat: 각 필터의 출력값을 이어 붙여 하나의 벡터로

#예제 6.33 합성곱 신경망 분류 모델 학습

- 최적화 함수: Adam 사용
 - Adam: 경사 하강법 알고리즘 개선 방법, 모멘텀 + RMSProp 결합 방법
 - 이전 기울기 값의 지수 이동 평균과 이전 기울기 값의 지수 이동 제곱 평균을 사용해 현재 기울기 값을 갱신
 - 새로운 기울기 값 반영 → 기울기가 빠르게 변화하는 구간에서 안정적인 수렴
 - 학습률을 조절해 발산 방지